



CORE COMPUTER SCIENCE

Database Management Systems

Placement Preparation Notes

Created by Pranav Gawai

Database Systems Roadmap

PLACEMENT PREPARATION GUIDE

How to use this Handbook:

Database Management Systems are tested heavily using transactional integrity questions (ACID, 2PL, Isolation Levels) and normalization mechanics (1NF to BCNF). Focus on index page structures, join optimization algorithms, and sharding vs replication scales.

Three-Phase Learning Plan

PHASE 1

Schema & Normalization

Master 3-tier architecture, functional dependencies, 1NF/2NF/3NF/BCNF normalization rules, and ER mappings. (Topics 1 – 6)

PHASE 2

Transactions & Locking

Study ACID properties, isolation levels (Dirty/Phantom reads), 2PL lock protocols, and deadlock avoidance models. (Topics 7 – 13)

PHASE 3

Queries, Indexing & SQL

Understand B+ Trees, relational algebra, joins, subqueries, materialized views, triggers, CAP theorem, and SQL exercises. (Topics 14 – 30)

Target Completion: 2.5 Hours Core Study & 30 Mins Self-Recall

Created by Pranav Gawai
grindos.pranavx.in

Table of Contents

Database Management Systems · Placement Preparation Handbook

DBMS ARCHITECTURE & FUNDAMENTALS

01	3-Tier Architecture	p.04
02	Data Independence	p.05

RELATIONAL MODEL CORE

03	Database Keys	p.06
04	Normal Forms (1NF → BCNF)	p.07
05	4NF & 5NF Normalization	p.08
22	SQL Constraints	p.25

DATABASE DESIGN & MODELS

06	ER Model to Relational	p.09
----	------------------------	------

TRANSACTION MANAGEMENT

07	ACID Properties & WAL	p.10
08	Transaction States	p.11
21	Database Recovery	p.24

CONCURRENCY & ISOLATION

09	Concurrency Anomalies	p.12
10	Conflict & View Serializability	p.13
11	Transaction Isolation Levels	p.14
12	2-Phase Locking (2PL)	p.15
13	DBMS Deadlocks	p.16

STORAGE & INDEXING

14	B-Tree vs B+ Tree	p.17
15	Static vs Dynamic Hashing	p.18

QUERY LANGUAGES & ALGEBRA

16	Relational Algebra	p.19
17	SQL Joins	p.20

SQL OPERATIONS

18	Subqueries & Correlated	p.21
19	Views vs Materialized Views	p.22
20	Triggers & Stored Procs	p.23

ADVANCED DBMS CONCEPTS

23	NoSQL vs SQL & CAP	p.26
24	Sharding & Replication	p.27

SQL PRACTICE PAGES

25	SQL: Nth Highest Salary	p.28
26	SQL: Duplicate Emails	p.29
27	SQL: Consecutive Numbers	p.30
28	SQL: Department Top Salaries	p.31
29	SQL: Exchange Seats	p.32
30	SQL: Cancellation Rate	p.33

DBMS ARCHITECTURE & FUNDAMENTALS

Yield: ★★★★★

01 – 3-Tier Architecture

The industry standard design structure for web applications.

DEFINITION & KEY CONCEPTS

- 1-Tier:** Direct user access to database (local dev).
- 2-Tier:** Client-Server model. Client contacts DB directly (JDBC/ODBC). Limited scalability.
- 3-Tier:** Adds an Application Server between client and database for business logic, safety, and caching.

Presentation Tier (UI)
Browser / Mobile App

↓↑ APIs / REST

Application Tier (Server)
Node.js / Python / Spring

↓↑ Connection Pooling

Data Tier (Database)
PostgreSQL / MySQL

CONNECTION POOLING

Rather than opening a new DB connection for every client request, the Application Server maintains a pool of active connections, recycling them to handle 10,000 clients with just 100 DB links.

WHERE USED IN INDUSTRY

Enterprise platforms (like Salesforce) implement 3-tier architecture to serve millions of clients, scaling app instances independently of core database clusters.

INTERVIEW DEPTH

- L1:** Client direct queries vs middleware orchestrations.
- L2:** Connection pool limits and socket recycling.
- L3:** Load balancing client sessions across application tiers.
- L4:** Security zoning and database firewall rules.
- L5:** Distributed caching strategies (Redis/Memcached) at application layer.

CORE INTERVIEW QUESTION

"Why is a 3-Tier architecture preferred over 2-Tier in enterprise web services?"

KEYWORDS TO DROP

DATA ABSTRACTION

CONNECTION POOLING

SECURITY FIREWALLS

INDEPENDENT SCALING

MODEL ANSWER

"A 3-Tier architecture provides data abstraction, preventing client devices from directly querying database engines. This shields credentials, mitigates SQL injections, allows connection pooling to conserve DB resources, and enables scaling server nodes independently of storage nodes."

DEEP DIVE FOLLOW-UP

→ **"Where does database caching typically live?"**
In the Application Tier (using in-memory stores like Redis) to serve read queries without hitting the database disk.

ARCHITECTURE DEPTH

- Level 1 (Basic):** Name Physical, Conceptual, and External layers.
- Level 2 (PDU):** Map user views to conceptual logical tables.
- Level 3 (Access):** Explain how the storage engine stores records.
- Level 4 (Cluster):** Detail distributed multi-node shard coordination layers.

COMMON MISTAKE

Confusing 3-schema abstraction with 3-tier system architecture.

TOP FOLLOW-UPS

- What is physical data independence?
- What is conceptual schema?

INTERVIEWER TRAP

Don't say 2-Tier is extinct. Many internal desktop software configurations and administrative shell tools still use direct 2-Tier database credentials.

MEMORY TRICK

3-Tier inserts an Application Server as a bouncer between the guest (client) and the VIP lounge (database).

02 – Data Independence

Modifying schema layers without cascading changes to higher levels.

SCHEMA ARCHITECTURE

Physical Independence: Changing physical storage organization (SSD migration, file pathing) without altering conceptual schemas.

Logical Independence: Changing conceptual table structures (adding columns/tables) without breaking external views or user queries.

```
[External Views] (Logical Independence)
  ↓ mapping layer
[Conceptual Schema] (Physical Independence)
  ↓ mapping layer
[Physical Storage Schema]
```

WHERE USED IN INDUSTRY

Cloud database migrations (e.g., migrating MySQL to AWS Aurora) rely on physical data independence, ensuring backend apps require zero code changes.

INTERVIEW DEPTH

- L1: Conceptual, logical, and physical schema mappings.
- L2: Physical independence (file layout changes).
- L3: Logical independence (view abstraction layers).
- L4: Query translation through schema mapping engines.
- L5: Managing schema migrations in blue-green deployments.

CORE INTERVIEW QUESTION

"If I create an index on a table, which type of data independence is demonstrated?"

KEYWORDS TO DROP

SCHEMA MAPPING

PHYSICAL FILES

QUERY ABSTRACTION

LOGICAL VIEWS

MODEL ANSWER

"Physical Data Independence. Adding an index changes how data is physically searched and stored on disk, but it does not change the logical tables, columns, or the SELECT queries used by the application code."

DEEP DIVE FOLLOW-UP

→ "Why is Logical Data Independence harder to achieve than Physical?"

Because changes to tables (like splitting one table into two) require complex view rewrites to present the original structure to applications.

REAL INTERVIEW SCENARIO

Interviewer: "How does logical data independence save dev time?"

Candidate: "It allows changing the conceptual schema (like splitting tables or adding fields) without having to rewrite user-mode application views or client SQL queries."

COMMON MISTAKE

Thinking physical schema changes require rewriting SQL queries.

TOP FOLLOW-UPS

- Differentiate logical and physical independence.
- How are mappings maintained?

INTERVIEWER TRAP

Don't confuse the two. If columns change, it is Logical. If disk file indexing changes, it is Physical.

MEMORY TRICK

Logical = Logic (tables/relations). Physical = Plastic/Metal (disks, file structures).

RELATIONAL MODEL CORE

Yield: ★★★★★

03 – Database Keys

Constraint identifiers to establish integrity and unique records.

KEY CLASSIFICATIONS

Super Key: Set of attributes that uniquely identifies a row. May contain redundant columns.

Candidate Key: A minimal Super Key. Removing any column destroys uniqueness.

Primary Key: The chosen candidate key. Cannot be NULL.

Foreign Key: References the Primary Key of another table, ensuring referential integrity.

[Super Keys (Broadest)]
↓ minimal subsets
[Candidate Keys]
↓ chosen one
[Primary Key]

WHERE USED IN INDUSTRY

Identity providers (like Okta) enforce strict primary and unique key constraints to ensure user email records never duplicate across login directories.

INTERVIEW DEPTH

- L1: Super keys, candidate keys, and primary keys.
- L2: Referential integrity constraints and foreign keys.
- L3: Composite keys vs auto-incrementing surrogate keys.
- L4: Cascade operations (ON DELETE CASCADE) internals.
- L5: UUID v4 vs sequential integer PK index fragmentation.

CORE INTERVIEW QUESTION

"Can a Foreign Key contain NULL values? Can a Primary Key contain NULLs?"

KEYWORDS TO DROP

ENTITY INTEGRITY

REFERENTIAL INTEGRITY

CANDIDATE KEY

NULL VALUE SUPPORT

MODEL ANSWER

"A Primary Key can never be NULL due to Entity Integrity, as it uniquely identifies the row. A Foreign Key, however, CAN be NULL if the relationship is optional (e.g. an employee without an assigned department)."

DEEP DIVE FOLLOW-UP

→ "What is a surrogate key?"

A system-generated key (like an auto-incrementing integer or UUID) with no business meaning, used to uniquely identify rows when natural keys are missing.

CONSTRAINTS DEPTH

Level 1 (Identify): Define primary keys and candidate keys.

Level 2 (Refer): Differentiate foreign key referential cascades vs restrictions.

Level 3 (Alternate): Identify alternate keys and unique constraints.

Level 4 (Surrogate): Explain compound natural vs single auto-increment surrogate keys.

COMMON MISTAKE

Thinking a table can have multiple primary keys. Only one primary key is allowed, but it can be composite.

TOP FOLLOW-UPS

- What is referential integrity?
- Can a primary key be null?

INTERVIEWER TRAP

Don't say a table can have multiple Primary Keys. It can only have ONE Primary Key, although that PK can be a composite of multiple columns.

MEMORY TRICK

Super = anything unique. Candidate = minimalist unique. Primary = the chosen candidate.

RELATIONAL MODEL CORE

Yield: ★★★★★

04 – Normal Forms (1NF → BCNF)

Structuring tables to eliminate redundancy and data anomalies.

NORMALIZATION RULES

- 1NF:** Atomic values only (no lists/arrays in a cell).
2NF: In 1NF + No Partial Dependencies (non-key columns must depend on the entire candidate key).
3NF: In 2NF + No Transitive Dependencies (non-key columns cannot depend on other non-key columns).
BCNF: For every dependency $X \rightarrow Y$, X must be a super key.

CORE INTERVIEW QUESTION

"Explain the difference between 3NF and BCNF. When does a table satisfy 3NF but fail BCNF?"

KEYWORDS TO DROP

- OVERLAPPING KEYS DETERMINANT CHECK
TRANSITIVE LOSS SUPER KEY RULE

MODEL ANSWER

"A table is in 3NF if for $X \rightarrow Y$, either X is a super key or Y is a prime attribute. BCNF removes the 'prime attribute' loophole. A table fails BCNF if it has overlapping candidate keys where a prime attribute transitively determines part of another key."

DEEP DIVE FOLLOW-UP

→ "Why do we intentionally denormalize database tables?"

To improve read performance in data warehouses or reporting services by avoiding expensive multi-table JOIN operations.

NORMALIZATION DEPTH

- Level 1 (Goal):** Prevent anomalies (insert, update, delete).
Level 2 (1NF/2NF): Atomize values; remove partial dependencies.
Level 3 (3NF): Remove transitive dependencies (non-key transitive rules).
Level 4 (Decompose): Validate lossless join and dependency preservation attributes.

Normal Form	Main Requirement
1NF	Atomic Columns
2NF	No Partial Dependencies
3NF	No Transitive Dependencies
BCNF	$X \rightarrow Y$ (X must be Super Key)

WHERE USED IN INDUSTRY

Financial ledgers and billing systems are normalized to 3NF or BCNF to prevent transaction anomalies and guarantee audit trail consistency.

INTERVIEW DEPTH

- L1:** Redundancy anomalies: insert, update, delete.
L2: 1NF atomic fields, 2NF partial dependencies.
L3: 3NF transitive dependencies, BCNF super key rules.
L4: Functional dependency closure and attribute sets.
L5: Lossless-join and dependency-preserving decompositions.

COMMON MISTAKE

Believing full normalization is always best. It causes multiple table joins, which degrades read performance.

TOP FOLLOW-UPS

- What is insertion anomaly?
- What is a prime attribute?

INTERVIEWER TRAP

Don't say BCNF is always preferred. BCNF can lose functional dependencies during decomposition, which is why 3NF is sometimes chosen.

MEMORY TRICK

The key, the whole key, and nothing but the key (so help me Codd).

RELATIONAL MODEL CORE

Yield: ★★★★★

05 – 4NF & 5NF Normalization

Handling multi-valued dependencies and join dependency structures.

HIGHER NORMAL FORMS

4NF: In BCNF + No Multi-Valued Dependencies (MVD). If A determines a set of values for B and C independently, B and C must not be stored in the same table.

5NF (PJNF): In 4NF + No Join Dependencies. A table cannot be reconstructed from several smaller tables without losing information.

WHERE USED IN INDUSTRY

Social network profiles use denormalized structures (4NF/5NF analysis) or NoSQL databases to store multi-valued attributes like user hobbies and friends.

INTERVIEW DEPTH

- L1:** Multivalued dependencies and 4NF rules.
- L2:** Join dependencies and 5NF (Project-Join Normal Form).
- L3:** Domain-Key Normal Form (DKNF) constraints.
- L4:** Denormalization trade-offs for read-heavy systems.
- L5:** Designing hybrid SQL-NoSQL schema layers.

CORE INTERVIEW QUESTION

"What is a Multi-Valued Dependency (MVD), and how does 4NF resolve it?"

KEYWORDS TO DROP

MULTI-VALUED DEP

INDEPENDENT ATTRIBUTES

TABLE DECOMPOSITION

TRIVIAL DEPENDENCY

MODEL ANSWER

"An MVD occurs when an attribute determines multiple independent values (e.g. a teacher teaches multiple subjects AND speaks multiple languages). 4NF resolves this by decomposing the table into separate tables: (Teacher, Subject) and (Teacher, Language)."

DEEP DIVE FOLLOW-UP

→ **"What is a Join Dependency in 5NF?"**

A join dependency exists if a table can be decomposed into N smaller tables, which when joined back, recreate the exact original table without spurious rows.

REAL INTERVIEW SCENARIO

Interviewer: "Why is a relation decomposed to BCNF instead of just 3NF?"

Candidate: "When there are multiple overlapping candidate keys. BCNF ensures that for every dependency $X \twoheadrightarrow Y$, X must be a super key, removing redundant insert anomalies that 3NF tolerates."

COMMON MISTAKE

Thinking all 3NF tables are in BCNF. BCNF is violated when a non-key attribute determines a part of a composite key.

TOP FOLLOW-UPS

- What is multi-valued dependency?
- Explain 4NF and 5NF.

INTERVIEWER TRAP

Don't worry about 5NF for simple business app interviews; it is extremely rare in real-world database design and mostly theoretical.

MEMORY TRICK

4NF splits unrelated checklists that get awkwardly crammed into a single table.

DATABASE DESIGN & MODELS

Yield: ★★★★★

06 – ER Model to Relational

Mapping Entity–Relationship diagrams to logical database tables.

MAPPING PARADIGMS

Strong Entity: Becomes a table with its key as the Primary Key.

Weak Entity: Becomes a table including its owner entity's PK as a Foreign Key, combined as a composite PK.

1-to-Many Relationship: Place the PK of the '1' side as a Foreign Key in the 'Many' side table.

Many-to-Many Mapping

Create a new **Junction Table** containing:
Composite PK: (FK_EntityA, FK_EntityB)

WHERE USED IN INDUSTRY

ORM tools (like Hibernate or Prisma) automatically map ER diagrams to SQL tables, generating primary–foreign key relationships from object schemas.

INTERVIEW DEPTH

L1: Entity sets, attributes, and relationships.

L2: Mapping strong vs weak entities to tables.

L3: Mapping 1:1, 1:N, and N:M relationships.

L4: Representing inheritance (Single Table vs Joined Table).

L5: Converting complex ER constraints into SQL assertions.

CORE INTERVIEW QUESTION

"How do you map a Weak Entity set to a relational table structure?"

KEYWORDS TO DROP

IDENTIFYING RELATIONSHIP

COMPOSITE PRIMARY KEY

DISCRIMINATOR COLUMN

CASCADE DELETE

MODEL ANSWER

"A weak entity cannot be identified by its own attributes alone. To map it: create a table, import the primary key of the identifying strong entity as a foreign key, and combine it with the weak entity's discriminator column as a composite primary key."

DEEP DIVE FOLLOW-UP

→ "What is a recursive relationship in ER diagrams?"

A relationship where an entity relates to itself (e.g. an **Employee table** has a Foreign Key pointing back to the **Employee table's Manager ID**).

ER SCHEMA DEPTH

Level 1 (Symbol): Map entities, attributes, and relationships.

Level 2 (Key): Map weak entities relying on identifying relationships.

Level 3 (Relation): Resolve many-to-many relationships into bridge tables.

Level 4 (Inherit): Map ISA specialization hierarchies (table-per-class vs table-per-hierarchy).

COMMON MISTAKE

Creating separate tables for 1:N relations instead of adding a foreign key to the 'Many' side table.

TOP FOLLOW-UPS

- Differentiate strong vs weak entities.
- What is participation constraint?

INTERVIEWER TRAP

Don't forget to implement CASCADE deletes for weak entity tables, as they cannot exist without their parent strong entities.

MEMORY TRICK

Many-to-Many = Marriage table. You need a third paper (junction table) linking the two spouses.

TRANSACTION MANAGEMENT

Yield: ★★★★★

07 – ACID Properties & WAL

Guarantees for reliable database transaction execution.

ACID GUARANTEES

Atomicity: All or nothing transaction execution.**Consistency:** Moves DB from one valid state to another (violating constraints rolls back).**Isolation:** Concurrent transactions run as if sequential.**Durability:** Committed data survives system crashes.

Write-Ahead Logging (WAL)

Transaction changes written to **WAL on Disk** first
↓ then
Data modified in buffer cache & flushed to DB disk

WHERE USED IN INDUSTRY

Payment gateways (like Stripe) use Write-Ahead Logging (WAL) to guarantee Durability; transactions are committed to disk logs before updating tables.

INTERVIEW DEPTH

- L1: Atomicity, Consistency, Isolation, and Durability.
- L2: Write-Ahead Logging (WAL) and commit protocols.
- L3: Buffer pool management and dirty page flushing.
- L4: Force/No-Force and Steal/No-Steal buffer policies.
- L5: Doublewrite buffers to prevent partial page write crashes.

CORE INTERVIEW QUESTION

"How does a database guarantee Atomicity and Durability during a sudden server crash?"

KEYWORDS TO DROP

WRITE-AHEAD LOG (WAL)

TRANSACTION ROLLBACK

REDO / UNDO LOG

FLUSH TO DISK

MODEL ANSWER

"Through Write-Ahead Logging (WAL). Every database modification is written to an append-only log on disk before changes are applied to data files. On crash recovery, the engine replays the WAL to REDO committed transactions (Durability) and UNDOs uncommitted ones (Atomicity)."

DEEP DIVE FOLLOW-UP

→ "What is a checkpoint in database logging?"

A marker in the log indicating that all dirty data buffers have been flushed to disk. During recovery, the DB only scans logs recorded after the last checkpoint.

RECOVERY DEPTH

Level 1 (ACID): Define Atomicity, Consistency, Isolation, and Durability.

Level 2 (WAL): Log edits on disk before dirtying cache pages.

Level 3 (Undo): Undo log allows rolling back incomplete transactions.

Level 4 (Redo): Redo log allows recovering committed data from crash states.

COMMON MISTAKE

Believing consistency is solely the database engine's job. It requires correct schema rules and user code logic.

TOP FOLLOW-UPS

- What is isolation level?
- How is durability achieved?

INTERVIEWER TRAP

Don't confuse ACID Consistency with CAP Theorem Consistency. ACID is constraint validation. CAP is distributed node synchronization.

MEMORY TRICK

WAL is like writing your will on paper before distributing the physical money.

TRANSACTION MANAGEMENT

Yield: ★★★★★

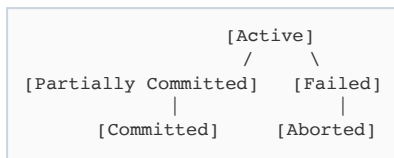
08 – Transaction States

The lifecycle stages of active database transactions.

STATE TRANSITIONS

Transactions traverse distinct states during execution:

Active (running queries) → **Partially Committed** (checks complete, but writing logs) → **Committed** (permanent) or **Failed/Aborted** (rollback).



WHERE USED IN INDUSTRY

High-throughput messaging systems (like RabbitMQ) track transaction states to ensure messages are either fully processed or safely rolled back.

INTERVIEW DEPTH

- L1: Active, partially committed, and committed states.
- L2: Failed and aborted transaction states.
- L3: Transaction rollback using rollback segments.
- L4: Savepoints and partial rollback implementations.
- L5: Distributed transaction state tracking (Two-Phase Commit).

CORE INTERVIEW QUESTION

"What is the difference between a Partially Committed state and a Committed state?"

KEYWORDS TO DROP

BUFFER CACHE

FLUSH TO DISK

COMMIT COMMAND

PERMANENT SAVE

MODEL ANSWER

"A transaction is partially committed when its final statement has run, but changes are still held in volatile RAM buffers. It transitions to the Committed state only after the commit records are written to the transaction log on disk."

DEEP DIVE FOLLOW-UP

→ **"What is a cascading rollback?"**

If Transaction A reads dirty data from uncommitted Transaction B, and Transaction B aborts, Transaction A must also be rolled back, potentially triggering a chain reaction.

REAL INTERVIEW SCENARIO

Interviewer: "What happens if a database crashes during a Partially Committed state?"

Candidate: "The state means execution finished but changes aren't flushed to disk. On reboot, the recovery manager reads WAL and rolls back the transaction because it never reached Committed."

COMMON MISTAKE

Believing 'Partially Committed' means data is saved. It just means statements finished running, but log buffer hasn't flushed.

TOP FOLLOW-UPS

- What is partially committed state?
- What triggers rollback?

INTERVIEWER TRAP

Don't assume a transaction is safe once the code completes. If the database crashes before the log is flushed, the transaction reverts to Aborted.

MEMORY TRICK

Partially Committed = checkout screen. Committed = receiving your payment receipt.

CONCURRENCY & ISOLATION

Yield: ★★★★★

09 – Concurrency Anomalies

Analyzing dirty reads, non-repeatable reads, and phantom reads.

ANOMALY DEFINITIONS

Dirty Read: Reading uncommitted data from another transaction that later aborts.

Non-Repeatable Read: Reading the same row twice within a transaction and getting different values because another transaction updated it.

Phantom Read: Re-executing a query with a range search and discovering new 'phantom' rows inserted by another transaction.

WHERE USED IN INDUSTRY

Inventory systems during flash sales manage concurrent writes using database locks to prevent selling the same item to two buyers.

INTERVIEW DEPTH

- L1:** Concurrency issues: dirty reads, non-repeatable reads, phantoms.
- L2:** Shared (S) vs Exclusive (X) lock request queues.
- L3:** Intention locks (IS/IX) in multiple granularity locking.
- L4:** Optimistic Concurrency Control (OCC) validation stages.
- L5:** Multi-Version Concurrency Control (MVCC) snapshot isolation.

CORE INTERVIEW QUESTION

"What is a Phantom Read, and how does it differ from a Non-Repeatable Read?"

KEYWORDS TO DROP

RANGE QUERY

ROW MODIFICATION

INSERT / DELETE

PREDICATE LOCKING

MODEL ANSWER

"A non-repeatable read occurs when an existing row's data is updated (a modification anomaly). A phantom read occurs when new rows are inserted or deleted within a queried range (a structural anomaly), violating range-query consistency."

DEEP DIVE FOLLOW-UP

→ **"What is a lost update anomaly?"**

When two transactions read the same value, modify it concurrently, and write back, with the second write overwriting the first change without incorporating it.

ANOMALIES DEPTH

- Level 1 (Conflict):** Understand read-write and write-write conflicts.
- Level 2 (Dirty):** Dirty Read (reading uncommitted transaction data).
- Level 3 (Repeat):** Non-repeatable Read (rereading rows yields altered values).
- Level 4 (Phantom):** Phantom Read (rereading queries yields inserted records).

COMMON MISTAKE

Thinking concurrency anomalies can only occur if the database crashes. They are scheduling order problems.

TOP FOLLOW-UPS

- What is dirty read?
- Explain phantom read.

INTERVIEWER TRAP

Don't assume all databases run in Serializable mode by default. Most default to Read Committed to prevent slow transactions.

MEMORY TRICK

Dirty = reading garbage. Non-Repeatable = values change under you. Phantom = new ghost rows appear.

CONCURRENCY & ISOLATION

Yield: ★★★★★

10 – Conflict & View Serializability

Evaluating if concurrent execution schedules match sequential execution.

CONFLICT CRITERIA

Two operations conflict if they belong to different transactions, target the same item, and at least one is a Write.

Conflict Serializability: A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

View Serializability: Stricter, focuses on final state and initial reads. All conflict serializable schedules are view serializable, but not vice versa.

WHERE USED IN INDUSTRY

Banking databases enforce serializability during account transfers to prevent race conditions and ensure double-spend protection.

INTERVIEW DEPTH

- L1: Conflict serializability vs view serializability.
- L2: Conflict equivalence and precedence graphs.
- L3: Serialization order and topological sorting.
- L4: Recoverable and cascade-less schedules.
- L5: Formal mathematical proof of serializability.

CORE INTERVIEW QUESTION

"How do you test a database schedule for Conflict Serializability?"

KEYWORDS TO DROP

PRECEDENCE GRAPH

CYCLE DETECTION

TOPOLOGICAL SORT

ACTIVE READ/WRITE

MODEL ANSWER

"Create a Precedence Graph where transactions are nodes. Draw a directed edge from T_i to T_j if T_i performs an operation that conflicts with T_j 's later operation. If the graph contains no cycles, the schedule is conflict serializable."

DEEP DIVE FOLLOW-UP

→ "What is a blind write?"

A write operation performed by a transaction on an item without reading it first. Blind writes allow a schedule to be view serializable without being conflict serializable.

REAL INTERVIEW SCENARIO

Interviewer: "How do you test if a schedule is conflict serializable?"

Candidate: "We build a precedence graph where transactions are nodes, and directed edges represent conflicting actions. If the precedence graph has no cycles, it is conflict serializable."

COMMON MISTAKE

Thinking view serializability is easy to compute. It is NP-complete. Conflict serializability is $O(V+E)$ via cycle check.

TOP FOLLOW-UPS

- What is precedence graph?
- Explain view serializability.

INTERVIEWER TRAP

Don't confuse scheduling with execution speed. Serializability ensures safety, not speed; serial schedules are highly inefficient in production.

MEMORY TRICK

Draw nodes for transactions. Connect conflicts. If a cycle forms, you have a collision.

CONCURRENCY & ISOLATION

Yield: ★★★★★

11 – Transaction Isolation Levels

Tuning safety vs performance inside ACID database engines.

ISOLATION OPTIONS

Defines how changes made by one transaction are visible to others. Enforces SQL-92 standards.

Level	Dirty Read	Non-Rep.	Phantom
Read Uncomm.	Allowed	Allowed	Allowed
Read Comm.	Prevented	Allowed	Allowed
Repeatable	Prevented	Prevented	Allowed
Serializable	Prevented	Prevented	Prevented

WHERE USED IN INDUSTRY

E-commerce checkout platforms tune isolation levels (e.g., Read Committed) to balance query performance with absolute data accuracy.

INTERVIEW DEPTH

- L1: ANSI SQL isolation levels: Read Uncommitted to Serializable.
- L2: Read phenomena: Dirty Read, Non-repeatable Read, Phantom Read.
- L3: Snapshot isolation internals and MVCC row versions.
- L4: Write skew anomaly under snapshot isolation.
- L5: Performance vs safety trade-offs in high-concurrency.

CORE INTERVIEW QUESTION

"Explain the trade-offs of the 'Serializable' isolation level compared to 'Read Committed'."

KEYWORDS TO DROP

LOCKING OVERHEAD

QUERY THROUGHPUT

CONCURRENCY CONTROL

OPTIMISTIC LOCKING

MODEL ANSWER

"Read Committed prevents dirty reads but allows non-repeatable reads, maximizing speed with minimal locks. Serializable guarantees complete isolation (preventing all anomalies) but introduces massive locking overhead, increasing transaction aborts and bottlenecks."

DEEP DIVE FOLLOW-UP

→ "How is Repeatable Read implemented?"

Typically using MVCC (Multi-Version Concurrency Control), where read queries access a static snapshot of the database taken when the transaction started.

SQL ISOLATION LEVELS

Read Uncommitted: Allows Dirty, Non-repeatable, and Phantom reads. No read locks.

Read Committed: Prevents Dirty reads. Reads read-committed data (shared locks released fast).

Repeatable Read: Prevents Dirty and Non-repeatable reads. Shared locks held until commit.

Serializable: Prevents all anomalies. Range locks acquired.

COMMON MISTAKE

Assuming default isolation is always Serializable. Most RDBMS use Read Committed or Repeatable Read.

TOP FOLLOW-UPS

- What is MVCC?
- What is snapshot isolation?

INTERVIEWER TRAP

Don't say Serializable runs transactions strictly one after the other. Modern databases run queries concurrently, aborting transactions only if conflict tests fail.

MEMORY TRICK

Higher isolation = more locks, slower throughput, maximum consistency safety.

CONCURRENCY & ISOLATION

Yield: ★★★★★

12 – 2-Phase Locking (2PL)

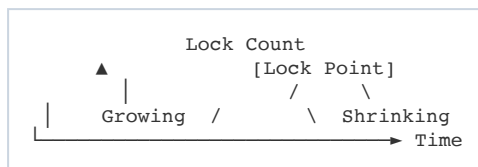
A lock acquisition protocol ensuring conflict serializability.

LOCKING PHASES

Growing Phase: Transaction acquires locks but cannot release any.

Shrinking Phase: Transaction releases locks but cannot acquire new ones.

Strict 2PL: Releasing exclusive (write) locks is delayed until the transaction commits or aborts, preventing cascading rollbacks.



CORE INTERVIEW QUESTION

"How does 2PL guarantee Conflict Serializability, and why is Strict 2PL preferred?"

KEYWORDS TO DROP

LOCK POINT

CASCADING ROLLBACK

EXCLUSIVE LOCK

GROWING/SHRINKING

MODEL ANSWER

"2PL ensures conflict serializability by preventing transactions from releasing a lock and acquiring a new one later, serializing transactions based on their Lock Point. Strict 2PL goes further by holding write locks until commit, preventing dirty reads and cascading rollbacks."

DEEP DIVE FOLLOW-UP

→ "Does 2PL prevent deadlocks?"

No. 2PL can easily lead to deadlocks if two transactions acquire locks in opposing order (e.g., T1 locks A, waits for B; T2 locks B, waits for A).

2PL LOCK DEPTH

Level 1 (Phase): Growing phase acquires locks; shrinking phase releases them.

Level 2 (Strict): Release exclusive locks at the very end (prevents cascading rollbacks).

Level 3 (Rigorous): Release all locks at transaction end.

Level 4 (Convert): Upgrade shared locks to exclusive; downgrade exclusive to shared.

WHERE USED IN INDUSTRY

Distributed database systems (like Google Spanner) use Strict 2-Phase Locking to prevent dirty reads and cascading transaction rollbacks.

INTERVIEW DEPTH

- L1: Growing phase, shrinking phase, and lock point.
- L2: Basic 2PL vs Conservative 2PL lock requirements.
- L3: Strict 2PL and Rigorous 2PL lock releases.
- L4: Cascading rollbacks prevention under Strict 2PL.
- L5: Mathematical proof of serializability under 2PL.

COMMON MISTAKE

Assuming basic 2PL prevents deadlocks. It only guarantees serializability. Deadlocks are still possible.

TOP FOLLOW-UPS

- Explain strict 2PL vs rigorous 2PL.
- What is lock conversion?

INTERVIEWER TRAP

Don't say 2PL is used in every relational database. Many modern engines use MVCC to avoid read locks entirely, using 2PL only for write operations.

MEMORY TRICK

Once you release a lock, you cannot ask for another. Acquire all keys first, then unlock.

CONCURRENCY & ISOLATION

Yield: ★★★★★

13 – DBMS Deadlocks

Handling circular lock dependencies inside transaction loops.

DEADLOCK MANAGEMENT

Prevention: Use lock ordering or timestamps (Wait-Die, Wound-Wait).

Detection: Maintain a Wait-For Graph (WFG) and scan for cycles periodically.

Recovery: Select a victim transaction (based on cost, age, or changes), abort it, and rollback its modifications.

WHERE USED IN INDUSTRY

High-concurrency SaaS backends run automated deadlock detection loops to select and terminate the cheapest transaction, releasing blocked threads.

INTERVIEW DEPTH

L1: Deadlock conditions: mutual exclusion, hold-wait, no-preemption, circular wait.

L2: Deadlock prevention using wait-die and wound-wait schemes.

L3: Deadlock detection using Wait-For Graphs (WFG).

L4: Victim selection criteria (rollback cost, lock hold duration).

L5: Distributed deadlock detection protocols.

CORE INTERVIEW QUESTION

"Compare the 'Wait-Die' and 'Wound-Wait' deadlock prevention schemes."

KEYWORDS TO DROP

TRANSACTION TIMESTAMP

PREEMPTION

WOUND (ABORT)

DIE (REVERT)

MODEL ANSWER

"Both schemes use transaction timestamps (older = higher priority). In **Wait-Die** (non-preemptive): if an older transaction needs a resource held by a younger one, it waits; if a younger needs from older, it dies. In **Wound-Wait** (preemptive): older wounds/aborts the younger; younger waits for older."

DEEP DIVE FOLLOW-UP

→ "Which is better: Wound-Wait or Wait-Die?"

Wound-Wait is generally more efficient because older transactions preemptively abort younger ones, minimizing idle waiting times.

REAL INTERVIEW SCENARIO

Interviewer: "How does wait-die differ from wound-wait deadlock prevention?"

Candidate: "Wait-die is non-preemptive: older waits, younger dies. Wound-wait is preemptive: older wounds/kills younger to take resource. Both prevent circular wait using transaction timestamps."

COMMON MISTAKE

Confusing Wait-Die (non-preemptive) vs Wound-Wait (preemptive). Wound-Wait is generally more efficient.

TOP FOLLOW-UPS

- Explain deadlock detection.
- What is wait-die scheme?

INTERVIEWER TRAP

Don't say you can avoid deadlocks entirely in high-concurrency systems. Deadlocks are normal; databases are designed to catch and retry aborted transactions.

MEMORY TRICK

Wait-Die = Older waits, younger dies. Wound-Wait = Older wounds, younger waits.

STORAGE & INDEXING

Yield: ★★★★★

14 – B-Tree vs B+ Tree

Balanced search trees optimized for secondary storage blocks.

STRUCTURE RULES

B-Tree: Stores keys, data pointers, and child pointers in both internal and leaf nodes.

B+ Tree: Stores keys and data pointers **only in leaf nodes**; internal nodes only store routing keys. Leaf nodes are linked sequentially.

B+ Tree Leaf Node Chain

[Leaf 1] → [Leaf 2] → [Leaf 3]
(Enables O(1) range scans along leaf links)

WHERE USED IN INDUSTRY

Search platforms use B+ Tree indexes to query millions of e-commerce products in milliseconds, bypassing expensive full table scans.

INTERVIEW DEPTH

- L1:** Clustered vs non-clustered index storage formats.
- L2:** B+ Tree index node splits and height balancing.
- L3:** Index scan, index seek, and covering index queries.
- L4:** Composite index prefix rules (leftmost column rule).
- L5:** Index fragmentation and rebuild/reorganize metrics.

CORE INTERVIEW QUESTION

"Why are B+ Trees preferred over B-Trees for database indexing?"

KEYWORDS TO DROP

INTERNAL NODE CAPACITY

FAN-OUT FACTOR

SEQUENTIAL SCAN

DISK I/O COUNT

MODEL ANSWER

"B+ Trees store data pointers only in leaf nodes. This makes internal nodes smaller, allowing them to fit more keys per disk block (a higher Fan-out Factor) and reducing tree height (fewer disk reads). Additionally, linked leaf nodes enable fast O(N) range queries."

DEEP DIVE FOLLOW-UP

→ **"What is a clustered index?"**

An index that dictates the physical storage order of rows on disk. A table can have only one clustered index (typically on the Primary Key).

INDEX STRUCTURE DEPTH

- Level 1 (Index):** Speeds searches; B+ Tree has linked leaf nodes for range queries.
- Level 2 (Cluster):** Define table physical row order (one per table).
- Level 3 (Non-Clust):** Secondary index containing key values and bookmarked data references.
- Level 4 (Cover):** SQL select attributes are fully contained inside index nodes.

COMMON MISTAKE

Believing indexing speeds up write queries (INSERT/UPDATE). It slows writes due to index structure updates.

TOP FOLLOW-UPS

- Why B+ Tree over B Tree?
- What is clustered index?

INTERVIEWER TRAP

Don't suggest binary search trees for database indexing. Disk lookups are slow; B+ trees use broad child pointers to minimize tree depth and disk I/O.

MEMORY TRICK

B+ Tree has the '+' sign for the linked leaf chain at the bottom.

STORAGE & INDEXING

Yield: ★★★★★

15 – Static vs Dynamic Hashing

Handling bucket overflow in database hash index files.

HASHING MECHANICS

Static Hashing: Maps keys to a fixed number of buckets. Overflow is handled using overflow chains (linked lists). Performance degrades over time.

Extendible (Dynamic) Hashing: Uses a directory of bucket pointers. Buckets split dynamically, adjusting directory depth as data grows.

WHERE USED IN INDUSTRY

In-memory key-value databases (like Redis) use hash tables for $O(1)$ lookups, providing ultra-fast session cache retrievals.

INTERVIEW DEPTH

- L1:** Static hashing vs dynamic hashing.
- L2:** Collision resolution: overflow chaining vs open addressing.
- L3:** Extendible hashing directories and bucket splits.
- L4:** Linear hashing load factors and split pointers.
- L5:** Hash index performance for range queries vs equality search.

CORE INTERVIEW QUESTION

"How does Extendible Hashing handle bucket overflow without rehashing all keys?"

KEYWORDS TO DROP

GLOBAL DEPTH

LOCAL DEPTH

DIRECTORY DOUBLING

BUCKET SPLIT

MODEL ANSWER

"Extendible hashing uses a directory of bucket pointers. Each bucket has a Local Depth, and the directory has a Global Depth. When a bucket overflows, only that bucket splits. If its local depth equals the global depth, the directory doubles in size, keeping other buckets untouched."

DEEP DIVE FOLLOW-UP

→ **"When is hash indexing preferred over B+ Tree indexing?"**

For point lookups (e.g. `WHERE ID = 5`), which hash indexes resolve in $O(1)$ time. However, they do not support range queries.

STATIC VS DYNAMIC HASHING

Static Hashing: Maps keys to fixed bucket counts. Fast but fails (causes overflows) if database size exceeds expectations.

Dynamic Hashing: Directory-based buckets that split/merge. Scales dynamically via directory doublings, preventing bucket overflow search degradation.

COMMON MISTAKE

Saying hashing is good for range queries. Hash indexes don't support sort order, rendering range searches useless.

TOP FOLLOW-UPS

- What is dynamic hashing directory?
- Differentiate static vs dynamic hashing.

INTERVIEWER TRAP

Don't use hash indexes if your application performs range queries (e.g. `WHERE age > 21`), as hash lookups are unordered.

MEMORY TRICK

Static = fixed slots (overflows crawl). Dynamic = directory doubles to fit new entries.

QUERY LANGUAGES & ALGEBRA

Yield: ★★★★★

16 – Relational Algebra

The formal procedural language defining relational operations.

FORMAL OPERATORS

Selection (σ): Filters rows based on a predicate condition (maps to SQL WHERE).

Projection (π): Selects specific columns, removing duplicates (maps to SQL SELECT).

Join ($\bowtie$): Combines tables based on a matching attribute condition.

```
 $\sigma_{age > 21} (\pi_{name, age} (Users))$   
(Filters rows after projecting columns)
```

WHERE USED IN INDUSTRY

SQL Query Optimizers translate SELECT statements into relational algebra trees to find the most efficient execution path.

INTERVIEW DEPTH

- L1: Select, project, and rename unary operators.
- L2: Union, intersection, set difference binary operators.
- L3: Cartesian product and join types (Theta, Natural, Outer).
- L4: Query tree parsing and relational equivalences.
- L5: Cost-based query optimization plans.

CORE INTERVIEW QUESTION

"What is the difference between Relational Algebra and Relational Calculus?"

KEYWORDS TO DROP

PROCEDURAL LANGUAGE

NON-PROCEDURAL / DECL.

TUPLE CALCULUS

OPERATOR FLOW

MODEL ANSWER

"Relational Algebra is procedural: it defines the step-by-step operators (Selection, Projection) used to retrieve data. Relational Calculus is non-procedural (declarative): it describes WHAT data to retrieve without specifying how to fetch it, similar to user-level SQL queries."

DEEP DIVE FOLLOW-UP

→ "What is the Cartesian Product ($\times$) operator?"

An operator combining all rows of Table A with all rows of Table B, producing $A \times B$ total rows. Forms the mathematical basis for JOIN operations.

RELATIONAL OPERATORS

Selection (σ): Filters table rows matching a predicate condition.

Projection (π): Filters table columns, discarding duplicates.

Cartesian Product ($\times$): Couples all row pairs of both relations.

Division ($\div$): Retrieves rows matching all records of divisor table.

COMMON MISTAKE

Confusing Selection (sigma σ) with SQL SELECT. σ filters rows (SQL WHERE). SQL SELECT matches Projection (π).

TOP FOLLOW-UPS

- Explain projection operator.
- What is natural join?

INTERVIEWER TRAP

Don't forget that projection (π) in relational algebra removes duplicates by default, unlike SQL SELECT which requires SELECT DISTINCT.

MEMORY TRICK

σ (sigma) = Selection (rows). π (pi) = Projection (columns).

17 – SQL Joins

Combining rows from two or more tables based on matching columns.

JOIN TAXONOMY

INNER: Returns matching rows from both tables.

LEFT: Returns all rows from the left table + matched rows from the right.

FULL: Returns all rows from both tables, filling mismatches with NULLs.

INNER

$A \cap B$
Only matches

LEFT

$A + (A \cap B)$
Keep Left



WHERE USED IN INDUSTRY

Analytics platforms join user activity tables with product catalogs to generate customized dashboard reports.



INTERVIEW DEPTH

L1: Inner join, left/right outer joins, full outer joins.

L2: Cross joins and self joins.

L3: Nested loop join, block nested loop join engines.

L4: Sort-merge join and partition-hash join algorithms.

L5: Join order optimizations in multi-table queries.

CORE INTERVIEW QUESTION

"What happens when you run an INNER JOIN on columns containing NULL values?"

KEYWORDS TO DROP

THREE-VALUED LOGIC

NULL EVALUATION

UNKNOWN MATCH

ROW EXCLUSION

MODEL ANSWER

"In SQL, NULL is evaluated as Unknown rather than a value. The comparison `NULL = NULL` evaluates to Unknown, not True. Therefore, an INNER JOIN excludes rows with NULL values in the join columns because they fail the predicate comparison."

DEEP DIVE FOLLOW-UP

→ **"What is a Self Join?"**

A join where a table is joined with itself by using alias names (e.g. matching employees to their managers in the same Employee table).



JOINS CONTRAST

Inner Join: Returns rows matching join condition on both sides.

Left Join: Returns all left rows plus matching right rows (nulls if no match).

Right Join: Returns all right rows plus matching left rows (nulls if no match).

Full Join: Returns all rows from both tables, padded with nulls where unmatched.



COMMON MISTAKE

Thinking Cross Join uses conditional matches. It evaluates the Cartesian Product (all row pairs).



TOP FOLLOW-UPS

- What is self join?
- Explain cross join.



INTERVIEWER TRAP

Don't confuse FULL OUTER JOIN with CROSS JOIN. FULL OUTER matches rows and fills gaps with NULLs. CROSS JOIN is a Cartesian product ($A \times B$ rows) with no matching condition.



MEMORY TRICK

Inner = matches only. Left = keep left side anyway. Full = keep both sides anyway.

SQL OPERATIONS

Yield: ★★★★★

18 – Subqueries & Correlated

Nested subqueries vs loops evaluated per outer row.

NESTED QUERIES

Subquery: A nested query executed once, with its output used by the outer query.

Correlated Subquery: A nested query referencing columns from the outer query. It is evaluated once for every row processed by the outer query, acting like a nested loop.

WHERE USED IN INDUSTRY

Reporting services write nested subqueries to identify top-performing sales representatives within each geographic region.

INTERVIEW DEPTH

- L1:** Correlated vs non-correlated subqueries.
- L2:** Subquery execution in SELECT, FROM, and WHERE clauses.
- L3:** Set comparison operators (IN, ANY, ALL, EXISTS).
- L4:** Query rewriting (subquery flattening to joins) by optimizer.
- L5:** Performance traps of correlated subqueries on large datasets.

CORE INTERVIEW QUESTION

"Why are Correlated Subqueries slow, and how can they be optimized?"

KEYWORDS TO DROP

ROW-BY-ROW LOOP

QUERY PLANNER

JOIN CONVERSION

EXISTS VS IN

MODEL ANSWER

"Correlated subqueries execute row-by-row, running the inner query once for each row in the outer query ($O(N^2)$ complexity). They can be optimized by rewriting them using JOINS or subqueries in the FROM clause, allowing the optimizer to process them as a single bulk operation."

DEEP DIVE FOLLOW-UP

→ **"When is EXISTS preferred over IN?"**

`EXISTS` stops scanning as soon as it finds a single match, making it faster than `IN` for large tables where the subquery returns many rows.

REAL INTERVIEW SCENARIO

Interviewer: "Why are correlated subqueries generally avoided?"

Candidate: "Because they reference outer query fields, meaning they must be re-evaluated once for every single row returned by the outer query, resulting in poor $O(N^2)$ execution."

COMMON MISTAKE

Using correlated subqueries in large tables without indexes. It causes performance bottlenecks ($O(N^2)$).

TOP FOLLOW-UPS

- Explain correlated subquery.
- Differentiate IN vs EXISTS.

INTERVIEWER TRAP

Don't assume subqueries are always slow. Modern query optimizers often rewrite simple subqueries as JOINS automatically.

MEMORY TRICK

Simple = run once, reuse result. Correlated = run for every row in the outer table.

SQL OPERATIONS

Yield: ★★★★★

19 – Views vs Materialized Views

Virtual queries vs physically cached query tables.

VIRTUALIZATION LAYERS

Standard View: A saved SQL query. Virtual only; it stores no physical data and runs the query each time it is accessed.

Materialized View: A physical table storing the query results on disk. Must be refreshed when underlying data changes.

Metric	Standard View	Materialized View
Storage	None (Saved Query)	Disk Storage
Execution	Runs on every access	Reads cached disk data
Latency	High (computed live)	Low (read direct)
Data Freshness	Real-time	Requires Refresh

WHERE USED IN INDUSTRY

Security layers use virtual read-only views to expose user names and emails while hiding sensitive hash passwords and credit cards.

INTERVIEW DEPTH

- L1: Virtual views vs materialized views.
- L2: View query rewriting and inline execution.
- L3: Updatable view rules (WITH CHECK OPTION).
- L4: Materialized view refresh strategies (incremental vs full).
- L5: Using views for column-level and row-level security.

CORE INTERVIEW QUESTION

"When would you choose a Materialized View over a Standard View?"

KEYWORDS TO DROP

ANALYTICAL CACHE

COMPLEX JOIN QUERY

REFRESH POLICY

READ PERFORMANCE

MODEL ANSWER

"Materialized views are preferred for expensive aggregation or JOIN queries on large datasets where real-time accuracy is not required (e.g. daily dashboards). Reading the pre-computed disk cache is much faster than running the query live."

DEEP DIVE FOLLOW-UP

→ **"What is incremental refresh in materialized views?"**

A refresh strategy that applies only the changes (inserts/updates) made since the last refresh, avoiding rebuilding the view from scratch.

VIEW EXECUTION DEPTH

Level 1 (Concept): Virtual view acts as a saved SELECT statement filter wrapper.

Level 2 (Updatable): View updates succeed only if mapped to single base table keys.

Level 3 (Materialize): Cache query outputs physically on disk to speed execution.

Level 4 (Refresh): Incrementally update materialized tables using trigger diffs.

COMMON MISTAKE

Modifying views expecting free writes. Only simple views with 1-to-1 table relations are writeable.

TOP FOLLOW-UPS

- What is materialized view?
- Can views be updated?

INTERVIEWER TRAP

Don't use materialized views for write-heavy tables with real-time read requirements; the refresh overhead will degrade performance.

MEMORY TRICK

Standard View = a shortcut link. Materialized View = downloading the file to your desktop.

SQL OPERATIONS

Yield: ★★★★★

20 – Triggers & Stored Procs

Automating database events and encapsulating business logic.

ACTIVE DATABASE OBJECTS

Trigger: An automated block executed in response to a database event (INSERT, UPDATE, DELETE).

Stored Procedure: Pre-compiled SQL code saved in the database, called explicitly to execute business logic.

Trigger Event hook example:

```
CREATE TRIGGER check_inventory
BEFORE UPDATE ON orders
FOR EACH ROW EXECUTE FUNCTION verify();
```

WHERE USED IN INDUSTRY

Compliance platforms implement database triggers to automatically write audit trail logs whenever user balance records are updated.

INTERVIEW DEPTH

- L1: Row-level vs statement-level trigger executions.
- L2: BEFORE vs AFTER trigger event firing sequences.
- L3: NEW and OLD pseudo-tables memory allocations.
- L4: Trigger cascading, recursion limits, and side-effects.
- L5: Trigger auditing performance costs on bulk updates.

CORE INTERVIEW QUESTION

"What are the disadvantages of relying heavily on database Triggers for business logic?"

KEYWORDS TO DROP

HIDDEN EXECUTION

DEBUGGING COMPLEXITY

PERFORMANCE OVERHEAD

CASCADE EXECUTION

MODEL ANSWER

"Heavy use of triggers hides execution logic from developers, making debugging difficult. Additionally, cascading triggers (where one trigger fires another) increase write latency and can lead to infinite loops or connection pool exhaustion."

DEEP DIVE FOLLOW-UP

→ **"What is the difference between a Stored Procedure and a Function?"**

Procedures cannot be called within SELECT queries and support transaction control (COMMIT/ROLLBACK). Functions must return a value and cannot manage transactions.

REAL INTERVIEW SCENARIO

Interviewer: "stored procedure vs database trigger: when to use which?"

Candidate: "Stored procedures are explicit blocks called manually by applications. Triggers are implicit blocks run automatically by the database on row modifications (INSERT/UPDATE/DELETE) for audit logging."

COMMON MISTAKE

Writing recursive trigger loops that lock database threads.

TOP FOLLOW-UPS

- Explain stored procedure vs trigger.
- Differentiate BEFORE vs AFTER trigger.

INTERVIEWER TRAP

Don't state that triggers are fast. While they run inside the database, they add overhead to every write operation.

MEMORY TRICK

Trigger = smoke detector (alarm goes off automatically). Procedure = calling the fire department manually.

TRANSACTION MANAGEMENT

Yield: ★★★★★

21 – Database Recovery

Restoring databases to a consistent state after system failures.

RECOVERY PARADIGMS

Log-Based Recovery: Replays logs (WAL) bottom-up, executing REDO and UNDO passes. Assures Durability.

Shadow Paging: Maintaining two page tables (active and shadow). Writing changes to new pages on disk, updating the active table pointer on commit.

WHERE USED IN INDUSTRY

Cloud storage backends use checkpointing and WAL logs to recover consistent database states after sudden hardware power failures.

INTERVIEW DEPTH

- L1:** Transaction log undo and redo pass execution.
- L2:** Log-based recovery algorithms (ARIES framework).
- L3:** Fuzzy checkpointing logs and dirty page tables.
- L4:** Compensation Log Records (CLRs) during undo loops.
- L5:** Recovering data consistency after media failure.

CORE INTERVIEW QUESTION

"Explain the difference between Shadow Paging and Log-Based Recovery."

KEYWORDS TO DROP

SHADOW PAGE TABLE

LOG FILE OVERHEAD

POINTER SWAP COMMIT

NO REDO / UNDO LOG

MODEL ANSWER

"Log-based recovery uses append-only logs on disk, requiring REDO/UNDO passes on restart. Shadow paging writes changes to new disk blocks and swaps a pointer to commit. It needs no logs or recovery passes, but causes disk fragmentation."

DEEP DIVE FOLLOW-UP

→ "What is the ARIES recovery algorithm?"

A popular log-based recovery algorithm with three phases: Analysis (finding active transactions), Redo (replaying all changes), and Undo (rolling back uncommitted changes).

RECOVERY LOGS DEPTH

Level 1 (Crash): Reboot triggers recovery engine to scan active log entries.

Level 2 (Check): Checkpoints write active dirty cache pages to disk, reducing log scan bounds.

Level 3 (Redo): Redo committed transactions to restore completed updates.

Level 4 (Undo): Undo active transactions to remove unfinished modifications.

COMMON MISTAKE

Thinking database recovery undoes every change. Commits are replayed (REDO), while uncommitted crashes are discarded (UNDO).

TOP FOLLOW-UPS

- What is deferred update?
- Explain checkpoint in recovery.

INTERVIEWER TRAP

Don't say shadow paging is standard. Modern relational databases (PostgreSQL, Oracle) use log-based WAL recovery due to better write performance.

MEMORY TRICK

Log-based = replaying history. Shadow Paging = editing a copy of a file and swapping the filenames on commit.

RELATIONAL MODEL CORE

Yield: ★★★★★

22 – SQL Constraints

Enforcing data integrity rules at the schema level.

CONSTRAINT TYPES

Constraints restrict the data values that can be inserted into tables, enforcing business logic at the database level.

UNIQUE: Enforces distinct values across rows (creates a B+ tree index automatically).

CHECK: Validates row values against a Boolean expression (e.g. `CHECK (age >= 18)`).

WHERE USED IN INDUSTRY

Medical record databases enforce CHECK and NOT NULL constraints to ensure patient vital stats fall within valid biological ranges.

INTERVIEW DEPTH

- L1:** Domain constraints, entity integrity, and referential integrity.
- L2:** CHECK, NOT NULL, and UNIQUE constraint checks.
- L3:** Deferred vs immediate constraint validation during commits.
- L4:** Enforcing cross-table assertions and complex constraints.
- L5:** System catalog metadata storage for constraints.

CORE INTERVIEW QUESTION

"How does a **UNIQUE** constraint differ from a **Primary Key** constraint?"

KEYWORDS TO DROP

MULTIPLE UNIQUE KEYS

NULL SUPPORT

CLUSTERED INDEX PK

NON-CLUSTERED UNIQUE

MODEL ANSWER

"A table can have only one Primary Key, which cannot contain NULLs and typically creates a clustered index. UNIQUE constraints allow multiple columns to be marked unique, support NULL values, and create non-clustered indexes."

DEEP DIVE FOLLOW-UP

→ "What is a **composite check constraint**?"

A **CHECK** constraint referencing multiple columns (e.g. `CHECK (sale_price < original_price)`), validating relationships between fields in the same row.

CONSTRAINTS DEPTH

Level 1 (Type): Map domains, keys, references, and check rules.

Level 2 (Entity): Primary keys must be unique and cannot contain NULL.

Level 3 (Referential): Foreign keys must match active parent primary keys.

Level 4 (Assertion): Multi-table constraints verified using catalog assertions.

COMMON MISTAKE

Forgetting ON DELETE CASCADE effects. Deleting a parent row will delete child rows automatically, which can cause mass data loss.

TOP FOLLOW-UPS

- What is domain constraint?
- What is check constraint?

INTERVIEWER TRAP

Don't assume a **UNIQUE** column cannot contain NULLs. In most databases, **UNIQUE** columns can contain multiple NULL values, as NULL is not equal to itself.

MEMORY TRICK

Constraints are the structural walls of your database, preventing bad data from entering.

ADVANCED DBMS CONCEPTS

Yield: ★★★★★

23 – NoSQL vs SQL & CAP

Relational storage vs horizontal scale-out architectures.

CAP THEOREM

In a distributed data store, you can guarantee at most two of: **Consistency** (all nodes see same data), **Availability** (every read succeeds), and **Partition Tolerance** (survives node drops).

Metric	SQL Databases	NoSQL Databases
Schema	Strict, Structured	Dynamic Schema
Scale	Vertical (bigger server)	Horizontal (more nodes)
Transaction	ACID compliant	BASE (Eventually consistent)
CAP Model	Typically CA	Typically AP or CP

WHERE USED IN INDUSTRY

Global shopping carts (like Amazon DynamoDB) prioritize Availability over Consistency (AP system) during peak traffic to never reject checkouts.

INTERVIEW DEPTH

L1: NoSQL categories: Key-Value, Document, Column-Family, Graph.
L2: CAP Theorem: Consistency, Availability, Partition Tolerance.
L3: BASE properties: Basically Available, Soft state, Eventual consistency.
L4: Vector clocks and conflict-free replicated data types (CRDTs).
L5: PACELC theorem extensions to CAP trade-offs.

CORE INTERVIEW QUESTION

"Why does the CAP Theorem force distributed databases to choose between Consistency and Availability?"

KEYWORDS TO DROP

NETWORK PARTITION

SYNC REPLICATION

STALE READ REJECT

EVENTUALLY CONSISTENT

MODEL ANSWER

"Network partitions (lost connections between nodes) are inevitable. If a partition occurs, the database must choose: either reject writes on isolated nodes to maintain Consistency (CP), or accept writes on all nodes, sacrificing Consistency for Availability (AP)."

DEEP DIVE FOLLOW-UP

→ "What are BASE properties in NoSQL?"

Basic Availability, Soft-state, and Eventual consistency. Replaces strict ACID guarantees to support horizontal scale.

CAP THEOREM DEPTH

Level 1 (Define): Consistency, Availability, Partition Tolerance.
Level 2 (Trade): Distributed partitions (P) force CP or AP options.
Level 3 (Consistency): CP yields strong consistency; AP yields eventual consistency.
Level 4 (NoSQL): Map Document (MongoDB), Key-value (Redis), Column (Cassandra) stores.

COMMON MISTAKE

Saying you can drop Partition Tolerance. Partitions are inevitable hardware errors. The real choice is Consistency vs Availability.

TOP FOLLOW-UPS

- Explain eventual consistency.
- Differentiate BASE vs ACID.

INTERVIEWER TRAP

Don't say SQL databases cannot scale. They can, but scaling horizontally requires complex replication and sharding setups.

MEMORY TRICK

SQL is a single bank vault (perfect consistency). NoSQL is distributed cash drawers (fast transactions, audited later).

ADVANCED DBMS CONCEPTS

Yield: ★★★★★

24 – Sharding & Replication

Distributing database workloads across multiple servers.

SCALE-OUT STRATEGIES

Replication: Copying the database across multiple servers. Master handles writes; replicas handle read queries (improves read capacity).

Sharding: Horizontal partitioning. Splitting a single table's rows across different database nodes based on a Shard Key (improves write capacity).

WHERE USED IN INDUSTRY

Social media giants (like Instagram) partition user tables by UserID (sharding) across database clusters to scale horizontally.

INTERVIEW DEPTH

- L1:** Vertical partitioning vs horizontal partitioning (sharding).
- L2:** Shard keys selection (range, hash, list sharding).
- L3:** Cross-shard queries and scatter-gather operations.
- L4:** Consistent hashing and shard rebalancing algorithms.
- L5:** Distributed transaction coordinations over shards.

CORE INTERVIEW QUESTION

"How does Sharding differ from Replication, and when is each chosen?"

KEYWORDS TO DROP

READ HEAVY REPLICATION

WRITE BOTTLENECK SHARD

PARTITIONING KEY

SHARED NOTHING

MODEL ANSWER

"Replication copies the entire database to multiple servers, optimizing read performance but keeping write bottlenecks on the master node. Sharding splits table rows across separate servers based on a shard key, dividing both read and write workloads across nodes."

DEEP DIVE FOLLOW-UP

→ "What is a hot shard?"

A shard receiving disproportionately high traffic due to a poor shard key choice (e.g. sharding by date, causing all recent writes to hit the same node).

REAL INTERVIEW SCENARIO

Interviewer: "Why does selecting the wrong shard key degrade performance?"

Candidate: "If we choose a biased key (like date), all inserts hit a single database node, causing hot-spotting. A good shard key distributes reads and writes uniformly across all nodes."

COMMON MISTAKE

Sharding too early. Try read-replicas, caching, and vertical hardware scaling first, since sharding complicates joins.

TOP FOLLOW-UPS

- Explain sharding key choice.
- Differentiate vertical vs horizontal partitioning.

INTERVIEWER TRAP

Don't suggest sharding unless database writes exceed a single server's hardware limits; it adds massive application complexity.

MEMORY TRICK

Replication = copying a book for multiple readers. Sharding = splitting a massive directory into letters A–M and N–Z.

SQL PRACTICE PAGES

Yield: ★★★★★

25 – SQL: Nth Highest Salary

Finding the Nth highest salary in an Employee table.

SQL QUERY CODE

```
SELECT DISTINCT Salary
FROM Employee
ORDER BY Salary DESC
LIMIT 1 OFFSET N-1;
```

ALTERNATIVE CTE METHOD (DENSE_RANK)

```
WITH RankedSalaries AS (
  SELECT Salary, DENSE_RANK()
    OVER (ORDER BY Salary DESC) as rk
  FROM Employee
)
SELECT Salary FROM RankedSalaries WHERE rk = N LIMIT 1;
```

WHERE USED IN INDUSTRY

HR payroll audits run dense ranking queries to identify employees with salaries in specific tax brackets.

INTERVIEW DEPTH

- L1: Using subqueries to find top N records.
- L2: DENSE_RANK() vs RANK() vs ROW_NUMBER() window functions.
- L3: Correlated subquery complexity ($O(N^2)$ search cost).
- L4: Index utilization in partition-ranking queries.
- L5: Performance comparison of offset queries vs subqueries.

CORE INTERVIEW QUESTION

"Why is DENSE_RANK() preferred over ROW_NUMBER() for the Nth highest salary query?"

KEYWORDS TO DROP

DUPLICATE SALARIES

DENSE VS GAP

RANKING WINDOW

ANALYTICAL QUERY

MODEL ANSWER

"ROW_NUMBER() assigns sequential integers to rows without handling duplicates, meaning duplicate highest salaries will take up ranks 1 and 2. DENSE_RANK() assigns the same rank to identical salaries without leaving gaps in the ranking order, ensuring accurate results."

DEEP DIVE FOLLOW-UP

→ "What is the difference between RANK() and DENSE_RANK()?"

RANK() leaves gaps after duplicate values (e.g. 1, 2, 2, 4). **DENSE_RANK()** does not leave gaps (e.g. 1, 2, 2, 3).

WINDOW FUNCTION DEPTH

RANK() OVER: Assigns ranks with gaps on salary ties (e.g., 1, 2, 2, 4).
DENSE_RANK() OVER: Assigns ranks without gaps on salary ties (e.g., 1, 2, 2, 3).
ROW_NUMBER(): Assigns sequential integers to rows (e.g., 1, 2, 3, 4).

COMMON MISTAKE

Writing `OFFSET N` instead of `OFFSET N-1`. OFFSET is 0-indexed.

TOP FOLLOW-UPS

- Explain DENSE_RANK() vs RANK().
- How to get 2nd highest salary?

INTERVIEWER TRAP

Don't use `LIMIT 1 OFFSET N` without subtracting 1; OFFSET is 0-indexed, so the 2nd highest salary requires `OFFSET 1`.

MEMORY TRICK

DENSE_RANK doesn't skip ranks; it keeps the numbering dense.

SQL PRACTICE PAGES

Yield: ★★★★★

26 – SQL: Duplicate Emails

Finding duplicate email records inside a Person table.

SQL QUERY CODE

```
SELECT Email
FROM Person
GROUP BY Email
HAVING COUNT(Email) > 1;
```

SELF-JOIN DELETION METHOD

```
DELETE p1 FROM Person p1, Person p2
WHERE p1.Email = p2.Email AND p1.Id > p2.Id;
(Deletes duplicate rows, keeping the smallest ID)
```

WHERE USED IN INDUSTRY

Marketing automation systems clean mailing lists by running duplicate checks to prevent sending spam emails to the same user.

INTERVIEW DEPTH

- L1: Finding duplicates using GROUP BY and HAVING clauses.
- L2: Deleting duplicates using SELF JOIN or CTID/ROWID.
- L3: Window functions (ROW_NUMBER) for duplicate isolation.
- L4: Database unique constraints to prevent duplicates.
- L5: Performance analysis of bulk duplicate deletes.

CORE INTERVIEW QUESTION

"Explain the difference between WHERE and HAVING clauses. Can we use aggregate functions in WHERE?"

KEYWORDS TO DROP

ROW FILTER (WHERE)

GROUP FILTER (HAVING)

AGGREGATE CONDITION

EXECUTION ORDER

MODEL ANSWER

"The WHERE clause filters rows before they are grouped (GROUP BY). The HAVING clause filters grouped rows after aggregation. Aggregate functions (like 'COUNT()') cannot be used in WHERE because it runs before grouping occurs."

DEEP DIVE FOLLOW-UP

→ "What is the query execution order in SQL?"
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

REAL INTERVIEW SCENARIO

Interviewer: "How do you delete duplicate emails using a self-join?"

Candidate: "We self-join the table on email and select the row with the larger ID to delete: 'DELETE p1 FROM Person p1 JOIN Person p2 ON p1.Email = p2.Email WHERE p1.Id > p2.Id'"

COMMON MISTAKE

Using 'WHERE' for aggregate counts. WHERE runs before rows are grouped, so you must use HAVING.

TOP FOLLOW-UPS

- How to delete duplicate rows?
- Explain self join deletion.

INTERVIEWER TRAP

Don't use HAVING without a GROUP BY clause, as it will evaluate the entire table as a single group, which is often not what you want.

MEMORY TRICK

WHERE filters individual rows. HAVING filters grouped blocks.

SQL PRACTICE PAGES

Yield: ★★★★★

27 – SQL: Consecutive Numbers

Finding numbers that appear at least three times consecutively.

SQL QUERY CODE

```
SELECT DISTINCT l1.Num as ConsecutiveNums
FROM Logs l1, Logs l2, Logs l3
WHERE l1.Id = l2.Id - 1
  AND l2.Id = l3.Id - 1
  AND l1.Num = l2.Num
  AND l2.Num = l3.Num;
```

OPTIMIZATION: LEAD / LAG WINDOW FUNCTIONS

```
SELECT DISTINCT Num FROM (
  SELECT Num,
    LEAD(Num, 1) OVER (ORDER BY Id) as next1,
    LEAD(Num, 2) OVER (ORDER BY Id) as next2
  FROM Logs
) t WHERE Num = next1 AND next1 = next2;
```

WHERE USED IN INDUSTRY

IoT monitoring dashboards run window functions to detect sensor failure anomalies when consecutive error signals are received.

INTERVIEW DEPTH

- L1:** Tracking consecutive sequences using SELF JOINS.
- L2:** Using LEAD() and LAG() window functions.
- L3:** Row number difference technique (Gaps and Islands).
- L4:** Time-series sequence tracking in SQL.
- L5:** Optimizing sequential range scans.

CORE INTERVIEW QUESTION

"Why are window functions (LEAD/LAG) preferred over self-joins for consecutive number queries?"

KEYWORDS TO DROP

SELF-JOIN OVERHEAD

INDEX SCAN LOOP

WINDOW FRAME PARTITION

SINGLE PASS SCAN

MODEL ANSWER

"Self-joins force the database to scan and join the logs table three times ($O(N^3)$ complexity without indexes). Window functions (LEAD/LAG) run in a single sequential pass over the sorted rows ($O(N \log N)$), reducing disk reads."

DEEP DIVE FOLLOW-UP

→ "What is the difference between LEAD and LAG?"
`LEAD` accesses data from subsequent rows. `LAG` accesses data from preceding rows within the same window frame.

ANALYTICAL WINDOW DEPTH

LEAD(col, 1): Fetches the value of the next row in the partition.

LEAD(col, 2): Fetches the value of the row two steps ahead.

LAG(col, 1): Fetches the value of the previous row. Use these to find consecutive numbers.

COMMON MISTAKE

Assuming ID sequences have no gaps. Always recommend creating row index sequences using ROW_NUMBER if gaps exist.

TOP FOLLOW-UPS

- Explain LEAD() and LAG() functions.
- How to find 3 consecutive rows?

INTERVIEWER TRAP

Don't assume ID values are sequential without gaps. If IDs have gaps, use `ROW\_NUMBER() OVER (ORDER BY Id)` to establish sequential row indexing first.

MEMORY TRICK

LEAD peeks forward. LAG peeks backward.

SQL PRACTICE PAGES

Yield: ★★★★★

28 – SQL: Department Top Salaries

Finding employees with the highest salaries in each department.

SQL QUERY CODE

```
SELECT d.Name as Department,
       e1.Name as Employee,
       e1.Salary
FROM Employee e1
JOIN Department d ON e1.DepartmentId = d.Id
WHERE e1.Salary = (
    SELECT MAX(Salary) FROM Employee e2
    WHERE e2.DepartmentId = e1.DepartmentId
);
```

WHERE USED IN INDUSTRY

Corporate resource planners execute partitioned rankings to calculate departmental salary distributions and budgets.

INTERVIEW DEPTH

- L1: Using window functions over partition groups.
- L2: `DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...)`.
- L3: Handling duplicate ranking boundary conditions.
- L4: Index design for partition window calculations.
- L5: Performance of CTEs vs nested rank subqueries.

CORE INTERVIEW QUESTION

"What is the difference between `DENSE_RANK()` and `MAX()` subqueries for department top salaries?"

KEYWORDS TO DROP

WINDOW PARTITIONING

CORRELATED SUBQUERY

DENSE RANK TIE

EXECUTION SPEED

MODEL ANSWER

"A `MAX()` correlated subquery runs row-by-row, which is slow. Using ``DENSE_RANK() OVER (PARTITION BY DepartmentId ORDER BY Salary DESC)`` partitions the data and ranks salaries in a single pass. Both handle ties correctly by returning all top-paid employees."

DEEP DIVE FOLLOW-UP

→ "What is the `PARTITION BY` clause?"

A window clause that groups rows into partitions before applying the window function (e.g. ranking salaries independently within each department).

PARTITIONING DEPTH

PARTITION BY: Divides query rows into groups (e.g., by department).
DENSE_RANK() OVER: Ranks salaries within each department partition separately.
Subquery filtering: Wraps rank calculation to filter rows where department rank ≤ 3 .

COMMON MISTAKE

Using ``ROW_NUMBER()``, which returns only one employee if there is a top-salary tie, violating department top list rules.

TOP FOLLOW-UPS

- Explain `PARTITION BY` clause.
- How to handle salary ties?

INTERVIEWER TRAP

Don't use ``ROW_NUMBER()`` if multiple employees share the top salary; it will randomly choose only one instead of returning all of them.

MEMORY TRICK

`PARTITION BY` is like `GROUP BY`, but it does not collapse rows.

SQL PRACTICE PAGES

Yield: ★★★★★

29 – SQL: Exchange Seats

Swapping adjacent seat IDs for consecutive students.

SQL QUERY CODE

```
SELECT
  CASE
    WHEN id % 2 = 1 AND id = (SELECT MAX(id) FROM
      Seat)
      THEN id
    WHEN id % 2 = 1
      THEN id + 1
    ELSE id - 1
  END as id, student
FROM Seat
ORDER BY id;
```

🏢 WHERE USED IN INDUSTRY

Booking systems swap seat numbers dynamically during reservation changes using CASE statements in SQL.

📊 INTERVIEW DEPTH

- L1:** Using CASE statements for dynamic value shifts.
- L2:** Swapping rows using arithmetic index offset logic.
- L3:** Handling odd-numbered row boundaries.
- L4:** LAG() and LEAD() window operations for seat changes.
- L5:** Transaction safety during dynamic updates.

CORE INTERVIEW QUESTION

"Explain how the CASE statement handles the final odd seat ID in this query."

KEYWORDS TO DROP

MAX BOUNDARY CHECK

MODULO ODD CHECK

SEQUENCE ORDER

CASE EVALUATION

MODEL ANSWER

"The first WHEN condition checks if the seat ID is odd and matches the maximum ID in the table. If true, the ID is left unchanged to prevent it from swapping to a non-existent higher seat. Other odd IDs are incremented, and even IDs are decremented."

DEEP DIVE FOLLOW-UP

→ "What is a COALESCE function?"

A function returning the first non-NULL value in its argument list. Used to handle empty or fallback values in queries.

💡 REAL INTERVIEW SCENARIO

Interviewer: "How do you swap adjacent seat IDs in SQL?"

Candidate: "Using a CASE statement on seat ID. If ID is odd and not last, add 1. If ID is even, subtract 1. If odd and last, keep ID: 'CASE WHEN id % 2 = 1 AND id = (select max(id) from seat) THEN id ...'"

⚠️ COMMON MISTAKE

Forgetting the 'ORDER BY id' clause at the end of the query, resulting in random output row sequences.

🔄 TOP FOLLOW-UPS

- How to swap adjacent seats?
- Explain CASE WHEN syntax.

🔴 INTERVIEWER TRAP

Don't forget to include the final 'ORDER BY id' clause; swapping IDs without sorting the output will return rows in incorrect order.

💡 MEMORY TRICK

Even becomes odd (subtract 1). Odd becomes even (add 1). Leave the final odd seat alone.

SQL PRACTICE PAGES

Yield: ★★★★★

30 – SQL: Cancellation Rate

Calculating trip cancellation rates for unbanned users.

SQL QUERY CODE

```
SELECT Request_at as Day,
       ROUND(SUM(CASE WHEN Status != 'completed' THEN 1
                     ELSE 0 END)
             / COUNT(*), 2) as "Cancellation Rate"
FROM Trips t
WHERE Client_Id NOT IN (SELECT Users_Id FROM Users
                        WHERE Banned = 'Yes')
   AND Driver_Id NOT IN (SELECT Users_Id FROM Users
                        WHERE Banned = 'Yes')
   AND Request_at BETWEEN '2013-10-01' AND '2013-10-03'
GROUP BY Request_at;
```

WHERE USED IN INDUSTRY

Ride-sharing dispatch services query taxi cancel rates over time intervals to penalize fraudulent driver accounts.

INTERVIEW DEPTH

- L1: Calculating percentages using filtered aggregate counts.
- L2: SUM(CASE WHEN ... THEN 1 ELSE 0 END) syntax patterns.
- L3: Handling division-by-zero boundary conditions.
- L4: Optimizing join-filter operations on massive transaction logs.
- L5: Time-series window filtering partitions.

CORE INTERVIEW QUESTION

"How does the SUM(CASE) construction calculate the cancellation rate in a single scan?"

KEYWORDS TO DROP

CONDITIONAL AGGREGATION

SUBQUERY FILTERING

RATIO ROUNDING

SINGLE PASS SCAN

MODEL ANSWER

"The `CASE` statement returns 1 if the trip status is cancelled (by client or driver) and 0 if completed. `SUM()` totals these 1s to get the count of cancelled trips. Dividing this by `COUNT(\*)` (total trips) yields the cancellation rate in a single scan."

DEEP DIVE FOLLOW-UP

→ "Why are NOT IN subqueries dangerous here?"

If the subquery returns even a single NULL value, the `NOT IN` condition will evaluate to Unknown, and the query will return zero rows. Using `NOT EXISTS` is safer.

AGGREGATION MATH DEPTH

Denominator: Total requests where clients/drivers are not banned.

Numerator: Requests cancelled by client or driver.

Division Rate: Use SUM(CASE WHEN status != 'completed' THEN 1 ELSE 0 END) / COUNT(*).

COMMON MISTAKE

Counting banned users because of missing filters in the WHERE block.

TOP FOLLOW-UPS

- How to calculate cancel rate?
- Explain date filtering.

INTERVIEWER TRAP

Don't count banned users. Always filter out banned clients and drivers in the WHERE clause first.

MEMORY TRICK

SUM(CASE WHEN cancelled THEN 1 ELSE 0 END) / COUNT(*) = Cancellation Rate.

DBMS Last-Minute Revision Sheet

Top Integrity Laws, Locking Schemes, and Normalization Guides

⚡ LOCKS & TX SUMMARY

ACID	Atomicity, Consistency, Isolation, Durability.
2-Phase Lock	Growing (locking) and shrinking (unlocking) stages.
Strict 2PL	No exclusive locks released until commit.
B+ Tree	Range queries supported; data stored in leaf nodes.
CAP Theorem	Consistency vs Availability under partitions.

🔧 NORMALIZATION LAWS

- **1NF:** Eliminate duplicate columns; enforce atomic values.
- **2NF:** Remove partial dependencies on candidate keys.
- **3NF:** Remove transitive dependencies of non-prime attributes.
- **BCNF:** For $X \rightarrow Y$, X must be a super key.
- **WAL:** Disk log write before memory DB page flush.

💡 TOP 5 INTERVIEW CONCEPTS TO RECALL

1. **Dirty vs Phantom reads:** Dirty = reading uncommitted data. Phantom = reading new rows added since.
2. **MVCC:** Multi-Version Concurrency Control prevents readers from blocking writers.
3. **Clustered vs Non-Clustered:** Clustered determines physical row storage order on disk (one per table).
4. **Wait-die vs Wound-wait:** Wait-die (older waits, younger dies). Wound-wait (older preempts younger).
5. **View vs Materialized:** View is query rewrite. Materialized view is query results cached on disk.

🎯 QUICK SELF-TEST CHECKLIST

☐ Differentiate 3NF and BCNF ☐ State isolation level anomalies ☐ Draw B+ Tree leaf link



DBMS Comparison Cheat Sheet

Quick Reference Contrast Tables for Fresher Interviews

PRIMARY KEY VS FOREIGN KEY

Feature	Primary Key	Foreign Key
Uniqueness	Must be unique	Duplicates allowed
Null Values	Strictly NOT NULL	NULL allowed
Table Limit	Max 1 per table	Multiple allowed
Purpose	Identify table records	Link related tables

CLUSTERED VS NON-CLUSTERED

Feature	Clustered	Non-Clustered
Physical Storage	Defines physical row order	Stored in separate structure
Leaf Nodes	Contain actual data rows	Contain pointers/keys
Table Limit	Max 1 per table	Multiple (e.g., 249+)
Lookup Steps	Direct retrieval	Bookmark lookup needed

3NF VS BCNF

Feature	3NF	BCNF
Dependency Rule	$X \rightarrow Y$, Y is prime ok	$X \rightarrow Y$, X must be super key
Strength	Weaker normalization	Stronger normalization
Overlapping Keys	Tolerated	Strictly normalized
Decomposition	Always preserves deps	May lose dependencies

VIEW VS MATERIALIZED VIEW

Feature	Virtual View	Materialized View
Storage	Query only (no disk space)	Saves query output on disk
Execution	Evaluates on every run	Precomputed; reads cached
Latency	Higher (joins evaluated)	Extremely low (flat read)
Updates	Directly reflects schema	Requires scheduled refresh

SQL VS NOSQL

Feature	SQL	NoSQL
Data Model	Relational Tables	Document, Key-value
Schema	Strict, static	Flexible, dynamic
Scaling	Vertical (bigger VM)	Horizontal (shards)
Transactions	Strong ACID	BASE / Eventual

TRIGGER VS STORED PROCEDURE

Feature	Trigger	Stored Procedure
Execution	Implicit on events (DML)	Explicit manual call
Parameters	No parameters supported	Accepts IN, OUT params
Return values	Cannot return values	Can return datasets
Typical Use	Auditing, constraints	Batch queries, complex business

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 1)

Q: What is database normalization, and explain the differences between 1NF, 2NF, 3NF, and BCNF?

"Normalization is the process of organizing database schemas to minimize data redundancy and prevent insertion, update, and deletion anomalies. In 1NF, we ensure all column values are atomic and each record is unique. In 2NF, we meet 1NF and eliminate partial dependencies, meaning every non-prime attribute must depend on the whole candidate key. In 3NF, we meet 2NF and eliminate transitive dependencies, meaning non-prime attributes cannot depend on other non-prime attributes. BCNF, or Boyce-Codd Normal Form, is a stronger version of 3NF where for every functional dependency $X \rightarrow Y$, X must be a super key. BCNF resolves anomalies that 3NF allows when there are overlapping candidate keys."

KEYWORDS: DATA REDUNDANCY ATOMIC VALUES PARTIAL DEPENDENCY TRANSITIVE DEPENDENCY SUPER KEY

Follow-Up: What is BCNF decomposition? Can a relation be in 3NF but not BCNF?

Common Mistake: Stating that 3NF allows transitive dependencies. It strictly forbids them.

Expected Depth: Detail the dependency preservation trade-offs when decomposing a relation to BCNF.

Q: Explain the ACID properties of a database transaction. How does the DBMS guarantee them?

"ACID represents Atomicity, Consistency, Isolation, and Durability, which guarantee reliable transaction processing. Atomicity ensures all operations in a transaction succeed, or the entire transaction is rolled back; this is guaranteed by the Undo Log and Write-Ahead Logging. Consistency ensures a transaction transitions the database from one valid state to another, enforcing constraints. Isolation ensures concurrent transactions execute without interfering with each other; this is guaranteed by concurrency control protocols like 2PL and lock-based isolation levels. Durability guarantees that once a transaction commits, its changes survive system crashes; this is achieved by flushing Redo Logs to non-volatile disk storage before acknowledging the commit."

KEYWORDS: TRANSACTION ROLLBACK WRITE-AHEAD LOGGING LOCK ISOLATION REDO LOGS CRASH RECOVERY

Follow-Up: Differentiate WAL vs Shadow Paging. What is the role of checkpoints in recovery?

Common Mistake: Believing Isolation is strictly handled by locks. Modern engines use MVCC (Multi-Version Concurrency Control) to avoid read-write blocking.

Expected Depth: Explain the commit protocols and write-amplification differences of log-structured merge trees vs B-trees.

Top Expected Interview Questions

Placement Q&As · Created by Pranav Gawai (Part 2)

Q: What is the difference between a Clustered and a Non-Clustered index? How do they affect read/write performance?

"A clustered index defines the physical order in which data rows are stored on disk. Because of this, a table can have only one clustered index, which is typically built on the primary key. Searching via a clustered index is extremely fast because the leaf nodes of the B+ tree contain the actual data rows. A non-clustered index is a separate structure from the table data; its leaf nodes contain the index key values and bookmark pointers, like a row ID or clustered key, pointing to the actual data location. Non-clustered indexes speed up secondary searches but require an extra lookup step to fetch the row data, and every additional index slows down write operations because all index trees must be updated during INSERT or UPDATE queries."

KEYWORDS: PHYSICAL ORDER LEAF NODES BOOKMARK LOOKUP PRIMARY KEY B+ TREE UPDATES

Follow-Up: What is a covering index? Why do databases use B+ trees instead of binary trees?

Common Mistake: Believing you can create multiple clustered indexes on a single table. The physical ordering constraint allows only one.

Expected Depth: Explain how B+ tree page splits occur during inserts and how fill-factors mitigate them.

Q: Compare SQL databases with NoSQL databases. When would you choose one over the other?

"SQL databases are relational, table-based, and enforce a strict, predefined schema with strong ACID properties, making them ideal for structured data and complex joins. NoSQL databases are non-relational, schema-less, and store data in flexible formats like documents, key-values, columns, or graphs. NoSQL databases typically trade strict consistency for high write throughput and horizontal scaling across clusters, often adhering to the CAP theorem's eventual consistency. I would choose SQL for systems requiring absolute data integrity, like transaction banking or inventory systems. I would choose NoSQL for unstructured, high-volume data streams like social feeds, real-time analytics, or rapid-growth application prototypes where schemas evolve hourly."

KEYWORDS: RELATIONAL SCHEMAS PREDEFINED CONSTRAINTS HORIZONTAL SCALING CAP THEOREM EVENTUAL CONSISTENCY

Follow-Up: Explain database sharding. Differentiate document stores from key-value stores.

Common Mistake: Thinking NoSQL databases are always faster. For complex relational queries, SQL databases are much faster and avoid manual application-layer joins.

Expected Depth: Detail CAP theorem proof and why you can only guarantee two out of Consistency, Availability, and Partition Tolerance in distributed systems.

DBMS Rapid Fire Questions

Fast-Recall Flashcards for Last-Minute Self-Testing

Q: What is a transaction?

A: A logical unit of work executing queries atomically.

Q: What does DDL stand for?

A: Data Definition Language (e.g. CREATE, DROP, ALTER).

Q: What does DML stand for?

A: Data Manipulation Language (e.g. SELECT, INSERT, UPDATE).

Q: What is a candidate key?

A: A minimal set of attributes that uniquely identifies a row.

Q: What is a surrogate key?

A: An artificial, system-generated primary key (like auto-increment ID).

Q: Difference between CHAR and VARCHAR?

A: CHAR is fixed-length (padded with spaces); VARCHAR is variable-length.

Q: What is a database view?

A: A virtual table representing the output of a saved SELECT query.

Q: What is a database trigger?

A: A set of SQL actions that execute automatically on INSERT, UPDATE, or DELETE.

Q: What is the CAP theorem?

A: A distributed database can guarantee at most two of: Consistency, Availability, and Partition Tolerance.

Q: What is write-ahead logging?

A: WAL requires recording changes in a log on disk before modifying the actual DB pages.

Q: What is database sharding?

A: Horizontal partitioning of database rows across multiple server instances.

Q: What is 2-Phase Locking (2PL)?

A: A lock protocol guaranteeing serializability with growing (locking) and shrinking (unlocking) phases.

DBMS Common Traps & Deflections

Tactical Responses to Deflect Tricky Placement Questions

Trap 1: The NULL Equality Comparison

Interviewer: "Why does `"SELECT * FROM table WHERE col = NULL"` return zero records even if NULL values exist?"

Deflection: Correct the syntax: In SQL, NULL represents an unknown state, not a value. Standard comparison operators like `=` or `!=` fail on NULL, returning NULL (unknown) rather than TRUE. You must use the `'IS NULL'` or `'IS NOT NULL'` operators to check for NULL values.

Trap 2: The MVCC Isolation Lock Myth

Interviewer: "Does isolation in transactional databases mean that reading a row always locks it?"

Deflection: Clarify that modern relational database engines use Multi-Version Concurrency Control (MVCC). Readers do not block writers, and writers do not block readers. The engine creates temporary versioned snapshots of the data, so read transactions query a point-in-time state without acquiring locks.

Trap 3: The 3NF vs BCNF Equivalence

Interviewer: "If a table is in 3NF and has no overlapping candidate keys, is it automatically in BCNF?"

Deflection: Yes. BCNF only differs from 3NF when there are overlapping candidate keys. If a table has only a single candidate key (or completely disjoint candidate keys) and is in 3NF, it is guaranteed to be in BCNF because no non-trivial dependencies on key subsets can exist.

Trap 4: Indexing Everything Performance

Interviewer: "To optimize database performance, shouldn't we add an index on every single column in the table?"

Deflection: Explain the massive drawback: While indexing speeds up read queries, it significantly degrades write performance (INSERT, UPDATE, DELETE) because the database must update the B+ tree index structure for every index on the table. It also consumes excessive disk space.

Trap 5: Trigger transactional scope

Interviewer: "If a trigger fails during an INSERT operation, does the original INSERT still commit?"

Deflection: No. Triggers execute within the same transactional context as the triggering statement. If the trigger raises an error or fails, the database rolls back the entire transaction, meaning the original INSERT is cancelled and not committed.

Personal Notes & Scribbles